

Integrated Systems Oncology: A Reproducible Computational Framework for Modeling Tumor Evolution, Metabolism, Epigenetics, and Adaptive Synergy

Dewan Sajid Islam 

Independent Researcher

Biam Model School and College

Dhaka, Bangladesh

Abstract

Cancer is a complex adaptive system whose progression and therapeutic response are shaped by coupled processes across genetic, metabolic, and epigenetic scales. Isolated computational models of any single axis of this system miss the interactions that determine emergent behaviors such as resistance and phenotypic plasticity. We present *Integrated Systems Oncology* (ISO), a modular and reproducible computational framework that integrates four independently validated engines: a Tumor Evolution Engine that models clonal population dynamics, mutation generation, and lineage structure through a discrete stochastic birth–death–mutation process; a Metabolism Engine that models reaction-diffusion dynamics of oxygen, glucose, lactate, ATP, pH, and reactive oxygen species together with the Warburg effect, mitochondrial efficiency, hypoxia, and necrosis; an Epigenetics Engine that models DNA methylation, histone acetylation, chromatin accessibility, gene-expression potential, stemness, plasticity, differentiation, and epigenetic age; and a Synergy Engine that aggregates the resulting fitness components through a systems-level state formulation with explicit feedback terms. ISO is implemented as four standalone Python packages under a master orchestration layer with engine-level and

project-wide validation. Each engine is deterministic given a random seed and is verified by dedicated validation suites that test numerical stability, the directionality of implemented relationships, and cross-run reproducibility. The framework itself, not any specific biological prediction, is the primary contribution of this work: it provides a transparent, extensible substrate for mechanistic computational experiments on interacting cancer-state variables. We describe the mathematical formulation, software architecture, validation architecture, and scope of ISO, and we explicitly separate computational synergy from clinical or experimental therapeutic synergy. All currently described capabilities are implemented and software-validated in the released software; capabilities that are not yet implemented are identified as future work. ISO is intended for reproducible computational oncology research, education, and scientific collaboration, and it is not claimed to be clinically validated.

Keywords: computational oncology, systems biology, cancer evolution, tumor metabolism, epigenetics, reproducible software, mathematical modeling, complex adaptive systems

1 Introduction

1.1 Cancer as a Complex Adaptive System

Cancer progression is increasingly understood as an evolutionary process in which heterogeneous cell populations compete for resources, adapt to microenvironmental pressures, and acquire resistance to therapeutic intervention (McGranahan and Swanton, 2017; Beerenwinkel et al., 2015). The underlying heterogeneity is multidimensional: genetic subclonal diversity, reversible epigenetic states, metabolic flexibility, and dynamic interactions with the tissue microenvironment together determine how a tumor responds to selection (Hanahan, 2022; Baylin and Jones, 2016; Pavlova et al., 2022). This multidimensionality is not merely a descriptive complication; it is the principal reason single-axis therapeutic and computational strategies tend to fail. When only one component of a complex adaptive system is targeted or modeled, the rest of the system adjusts, and the perturbation is absorbed by compensatory dynamics.

Computational modeling has become a central methodology for studying such systems (Kitano, 2002). Population-dynamics models of clonal evolution, reaction-diffusion models of tumor metabolism, and dynamical models of epigenetic regulation have each yielded valuable insight (Altrock et al., 2015; Markert and Vazquez, 2015). However, these models are typically developed in isolation, with different state spaces, different time scales, and different assumptions about how the components of the system couple.

This fragmentation limits the kinds of questions that can be asked: interactions between evolutionary, metabolic, and epigenetic processes are exactly where much of the biologically interesting behavior occurs, yet they are difficult to study when each process is captured only in its own separate framework.

1.2 The Case for a Modular Systems Framework

There are two broad strategies for addressing this fragmentation. One is to build a single monolithic simulator that attempts to represent all relevant biology within a single coupled formalism. This approach maximizes immediate integration but tends to sacrifice interpretability, modularity, and verifiability: it becomes difficult to isolate which submodel is responsible for a given behavior and difficult to validate the submodels independently. The alternative, which we adopt here, is to build a set of independent, carefully validated engines that each represent one clearly bounded biological domain, designed to be coupled through explicit, documented interfaces at the systems level. This second approach accepts the cost of designing coupling interfaces but gains the substantial benefits of independent development, independent validation, and independent reuse.

We present *Integrated Systems Oncology* (ISO), a computational framework built on this second principle. ISO comprises four engines—Tumor Evolution, Metabolism, Epigenetics, and Synergy—each implemented as a standalone Python package with its own configuration, state variables, update rules, and validation suite. A master orchestration layer executes the engines in sequence and reports their status. A project-wide validation script executes the validation suite of every engine and produces a unified report. Each engine is deterministic given a random seed, and reproducibility across runs is verified by dedicated validators.

1.3 Scope and Scientific Contribution

The contribution of this work is methodological. ISO is not presented as a clinically validated tool, nor does this manuscript report empirical biological discoveries. Instead, ISO is presented as a reproducible computational substrate for mechanistic experiments on interacting cancer-state variables, in which each engine implements a well-defined subset of the biology and the Synergy Engine implements a systems-level aggregation of the resulting state. The framework’s scope is deliberately bounded: it currently represents clonal population dynamics and mutation bookkeeping, metabolic reaction-diffusion dynamics with the Warburg effect and necrosis, dynamic chromatin and differentiation state, and systems-level adaptive properties such as mutation pressure, therapy resistance, adaptive capacity, resilience, and stability. It does not currently represent the immune system, therapeutic intervention, spatial three-dimensional tissue architecture, or patient-specific calibration.

The manuscript is organized as follows. Section 2 introduces the four engines and their state spaces. Section 3 presents the mathematical formulation of the model as implemented. Section 4 describes the software architecture and orchestration. Section 5 describes validation and reproducibility. Section 7 reports framework-level results that are supported by the released software, explicitly labeled as computational and methodological rather than empirical. Section 8 discusses interpretation, limitations, and the distinction between computational and clinical synergy. Section 9 outlines future work.

Table 1: Overview of the four ISO engines.

Engine	Domain	Representative State Variables	Coupling Role
Tumor Evolution	Clonal population dynamics, mutation, lineage	clone populations, growth rates, mutation counts, fitness, generation	Provides per-clone evolutionary state
Metabolism	Reaction-diffusion dynamics of the tumor microenvironment	oxygen, glucose, lactate, ATP, pH, ROS, hypoxia, necrosis, phenotype	Provides per-region metabolic state
Epigenetics	Dynamic chromatin and differentiation state	methylation, acetylation, chromatin accessibility, expression potential, stemness, plasticity, differentiation, epigenetic age	Provides per-region epigenetic state
Synergy	Systems-level adaptive integration	combined fitness, mutation pressure, resistance, adaptive capacity, resilience, stability	Aggregates evolutionary, metabolic, and epigenetic fitness

2 The Integrated Systems Oncology Framework

ISO consists of four computational engines, each modeling a distinct biological domain, together with a master orchestration layer and a project-wide validation suite. This section describes the four engines and the state spaces they represent. The mathematical formulation of the update rules is presented in Section 3.

2.1 Tumor Evolution Engine

The Tumor Evolution Engine models clonal population dynamics with heritable phenotypic state through a discrete stochastic birth–death–mutation process. Its state consists of a collection of clones, each characterized by a population size, an intrinsic growth rate, a mutation rate, a set of heritable phenotype traits, and lineage metadata.

The engine implements:

- **Clonal growth:** Population dynamics with density-dependent competition. Birth and death events are drawn stochastically for each clone at each step; net population change emerges from the difference between realized births and deaths.
- **Mutation generation:** Mutation opportunities are generated from division activity.

Each opportunity is classified stochastically as driver or passenger and, if it establishes, produces a new clone whose phenotype is inherited from its parent with modification.

- **Selection:** Clones with higher effective birth relative to death expand more rapidly; selection is therefore an emergent property of the underlying stochastic process rather than an explicitly applied replicator operator.
- **Lineage tracking:** Every clone records a parent identifier, a generation depth, and a root ancestor identifier, enabling reconstruction of the clonal lineage tree.
- **Diversity metrics:** Shannon and Simpson diversity indices are computed from the clone population distribution.
- **Reproducibility:** All stochastic events are generated from a seeded NumPy random generator, ensuring identical simulations for identical configurations.

The engine's purpose is to represent the evolutionary state of a heterogeneous tumor: which clones exist, how their populations change over time, how new clones arise through mutation, and how selection reshapes the population distribution.

2.2 Metabolism

Engine

The Metabolism Engine models the spatiotemporal dynamics of key metabolites across a one-dimensional grid of tissue regions using a reaction-diffusion formulation. Its state consists of per-region concentrations of oxygen, glucose, lactate, ATP, pH, and reactive oxygen species, together with per-region discrete metabolic phenotypes and a necrosis flag. The engine implements:

- **Diffusion:** Fickian diffusion of oxygen, glucose, lactate, and pH across regions, with zero-flux boundaries except at vascular regions where concentrations are fixed by supply.
- **Consumption and production:** Cellular consumption of oxygen and glucose, production of ATP through glycolysis and oxidative phosphorylation, production of lactate from glycolysis, and production of reactive oxygen species from oxidative metabolism.
- **Warburg effect:** A configurable bias toward aerobic glycolysis, parameterized by a Warburg bias and an aerobic glycolysis fraction, so that glycolytic flux persists even under oxygen-replete conditions.
- **Mitochondrial efficiency:** A configurable scaling of oxidative ATP production.
- **pH dynamics:** Acidification by lactate and buffering back toward a physiological baseline.

- **ROS dynamics:** Production coupled to oxidative metabolism and first-order decay.
- **Necrosis:** Regions enter a necrotic state when ATP falls below a threshold and stress remains elevated for a sustained interval; necrotic regions are metabolically inactive.
- **Phenotype assignment:** Each region is assigned to one of six metabolic phenotypes (oxidative, glycolytic, intermediate, hypoxic, quiescent, necrotic) using explicit threshold rules.

The engine's purpose is to represent the metabolic microenvironment as a spatially resolved field, providing the metabolic context in which clonal populations evolve.

2.3 Epigenetics

Engine

The Epigenetics Engine models dynamic chromatin and differentiation state across the same one-dimensional spatial grid. Its state consists of per-region values for DNA methylation, histone acetylation, chromatin accessibility, gene-expression potential, epigenetic instability, plasticity, stemness, differentiation, environmental stress, and epigenetic age. The engine implements:

- **Methylation dynamics:** Passive drift toward an equilibrium, active demethylation promoted by acetylation, and stress-dependent modulation.
- **Acetylation dynamics:** Acetylation and deacetylation with stress-dependent modulation.
- **Chromatin accessibility:** Opening driven by acetylation and closing driven by methylation.
- **Gene-expression potential:** A linear combination of acetylation, methylation, and accessibility, with stochastic noise.
- **Instability and plasticity:** Instability increases with stress and decays; plasticity increases with instability and decays.
- **Stemness and differentiation:** Coupled dynamics in which stemness declines with differentiation and increases with plasticity, and differentiation progresses toward completion with plasticity permitting partial dedifferentiation.
- **Epigenetic age:** Accumulation over time with partial reset by high plasticity.
- **Discrete classification:** Each region is assigned a discrete differentiation state and a discrete chromatin state.

The engine's purpose is to represent the epigenetic and differentiation state of the tumor as a dynamic field, capturing how reversible regulatory states can shift under environmental pressure.

2.4 Synergy

Engine

The Synergy Engine provides systems-level integration of the evolutionary, metabolic, and epigenetic fitness components. It is initialized with estimates of tumor, metabolic, and epigenetic fitness drawn from configuration parameters, and it evolves its own systems-level state under explicit update rules. Its state consists of per-region values for tumor fitness, metabolic fitness, epigenetic fitness, combined fitness, stemness, plasticity, mutation pressure, selection pressure, metabolic stress, oxidative stress, hypoxia, chromatin accessibility, gene expression, adaptive capacity, therapy resistance, cellular resilience, and system stability. The engine implements:

- **Combined fitness:** A weighted sum of tumor, metabolic, and epigenetic fitness.
- **Stress dynamics:** Hypoxia, oxidative stress, and metabolic stress evolving with bounded production and decay terms.
- **Mutation pressure:** Driven by metabolic and oxidative stress, with saturating production and decay.
- **Therapy resistance:** Driven by plasticity and resilience, with saturating production and decay.
- **Adaptive capacity:** Driven by gene expression, with saturating production and decay.
- **Cellular resilience:** Increased by stemness and decreased by stress.
- **System stability:** Increased by resilience and decreased by oxidative stress and mutation pressure.
- **Negative feedback:** Resistance reduces selection pressure, and mutation pressure reduces epigenetic fitness, preventing runaway positive feedback.

The term “synergy” in ISO refers to computational or model-level interaction between state variables, not to a clinically or experimentally demonstrated therapeutic synergy.

This distinction is emphasized throughout the manuscript.

3 Mathematical

Formulation

This section presents the mathematical formulation of the engines as implemented.

Notation is defined where first introduced and used consistently thereafter. The numerical treatment differs across engines: the Tumor Evolution Engine is a discrete stochastic process, while the Metabolism, Epigenetics, and Synergy Engines are

updated by explicit time-stepping of continuous state variables. Bounds on state variables depend on the physical meaning of each variable: normalized state variables are constrained to $[0, 1]$; concentrations are constrained to be non-negative; pH is constrained to a physical range; populations are non-negative and unnormalized; and categorical variables take discrete values from a finite set.

3.1 Tumor Evolution Dynamics

Let P_i denote the population of clone i , and let $N = \sum_j P_j$ denote the total tumor burden. The evolution of each clone proceeds through discrete stochastic birth and death events within each time step Δt .

Birth process. The effective per-capita birth rate of clone i is

$$b_i = b_0 \phi_i \eta_i \cdot \max\left(0, 1 - \frac{N}{K}\right), \quad (1)$$

where b_0 is the base birth rate, ϕ_i the clone's proliferation capacity, η_i its metabolic efficiency, and K the carrying capacity. The realized number of births in a step is drawn from a Poisson distribution,

$$B_i \sim \text{Poisson}(b_i P_i \Delta t). \quad (2)$$

Death process. The effective per-capita death rate of clone i is

$$d_i = d_0 \alpha_i (1 + \nu_i), \quad (3)$$

where d_0 is the base death rate, α_i the clone's apoptosis susceptibility, and ν_i its immune visibility. The realized number of deaths in a step is

$$D_i \sim \text{Poisson}(d_i P_i \Delta t). \quad (4)$$

Population update. Populations are updated as

$$P_i \leftarrow \max(0, P_i + B_i - D_i). \quad (5)$$

Selection is therefore not an explicitly applied operator; clones with higher effective birth relative to death expand more rapidly under the same competition and mutation regime.

Mutation generation. Mutation opportunities arise from cell division. The expected number of mutation opportunities generated by clone i in a step is

$$\lambda_i = B_i \mu_i, \quad (6)$$

where μ_i is the effective per-division mutation rate,

$$\mu_i = \mu_0 \cdot \frac{1}{\max(\rho_i, \rho_{\min})} \cdot (1 + \iota_i), \quad (7)$$

with μ_0 the base rate, ρ_i the DNA repair efficiency, $\rho_{\min}$ a numerical floor, and ι_i the genomic instability. Realized attempts are drawn as

$$M_i \sim \text{Poisson}(\lambda_i). \quad (8)$$

Each attempt is stochastically classified as a driver mutation with probability p_{driver} , otherwise as a passenger mutation. Each attempt then establishes a new lineage with probability p_{est} . Successful establishments consume a small seed population from the parent clone and produce a child clone whose phenotype is inherited and modified. The effect of a mutation on growth-related traits is sampled from a zero-mean normal distribution, with a larger standard deviation used for driver mutations and a smaller standard deviation used for passenger mutations, so that drivers have larger typical effect magnitudes than passengers while both remain signed.

Fitness as an observable. Fitness is not imposed exogenously but derived from the realized birth and death rates,

$$f_i = b_i - d_i, \quad (9)$$

and is recorded as a diagnostic quantity. Diversity of the clone population is measured by the Shannon index

$$H = - \sum_i \frac{P_i}{N} \ln \frac{P_i}{N}, \quad (10)$$

and the Simpson index

$$D = 1 - \sum_i \left(\frac{P_i}{N} \right)^2. \quad (11)$$

3.2 Metabolic

Dynamics

The Metabolism Engine uses a one-dimensional spatial grid indexed by $j = 1, \dots, n$.

Concentrations are updated by explicit time-stepping.

Diffusion. Let $C_j^{(k)}$ denote the concentration of metabolite k in region j . Diffusion is modeled with a discrete Laplacian,

$$\mathcal{D}_j^{(k)} = D_k \left(C_{j-1}^{(k)} - 2C_j^{(k)} + C_{j+1}^{(k)} \right), \quad (12)$$

with zero-flux boundaries at both ends and fixed-concentration boundary conditions at vascular regions.

Consumption. Oxygen and glucose consumption for a non-necrotic region are

$$O_j^{\text{cons}} = k_{O_2} C_j^{O_2} (1 - g_j) \Delta t, \quad (13)$$

$$G_j^{\text{cons}} = k_G C_j^G (1 + g_j) \Delta t, \quad (14)$$

where g_j is the glycolytic fraction of metabolism in that region.

Warburg effect. The glycolytic fraction is

$$g_j = \text{clip} \left(w + (1 - w)(1 - \tilde{O}_j) + a, 0, 1 \right), \quad (15)$$

where w is the Warburg bias, a the aerobic glycolysis fraction, and $\tilde{O}_j$ the normalized oxygen concentration. This captures aerobic glycolysis even under oxygen-replete conditions.

ATP production.

$$\text{ATP}_j^{\text{prod}} = G_j^{\text{cons}} g_j y_g + G_j^{\text{cons}} (1 - g_j) y_o \epsilon, \quad (16)$$

where y_g and y_o are the glycolytic and oxidative ATP yields and ϵ is the mitochondrial efficiency.

Lactate, ROS, pH. Lactate production is

$$L_j^{\text{prod}} = G_j^{\text{cons}} g_j k_L. \quad (17)$$

Reactive oxygen species are produced in proportion to oxidative ATP production and decay at first order,

$$\frac{d\text{ROS}_j}{dt} = k_{\text{ROS}} \text{ATP}_j^{\text{OXPHOS}} - \kappa_{\text{ROS}} \text{ROS}_j. \quad (18)$$

pH evolves through lactate-driven acidification and buffering back toward a physiological baseline,

$$\frac{d\text{pH}_j}{dt} = -k_{\text{acid}} L_j + k_{\text{buff}} (\text{pH}_0 - \text{pH}_j). \quad (19)$$

Necrosis and phenotypes. A region is marked necrotic when its ATP falls below a threshold θ_{ATP} and its stress exceeds a threshold θ_{stress} for a sustained number of consecutive steps; necrotic regions become metabolically inactive. Discrete metabolic phenotypes (oxidative, glycolytic, intermediate, hypoxic, quiescent, necrotic) are assigned by threshold rules applied to oxygen, ATP, and stress.

3.3 Epigenetic

Dynamics

Let Met_j , Ace_j , and Acc_j denote methylation, acetylation, and chromatin accessibility in region j , and let S_j denote environmental stress. The implemented update rules are

$$\frac{d\text{Met}_j}{dt} = k_M^{\text{drift}} (M_{\text{eq}} - \text{Met}_j) + k_M^{\text{demeth}} \text{Ace}_j (1 - \text{Met}_j) + k_M^{\text{stress}} S_j (0.5 - \text{Met}_j), \quad (20)$$

$$\frac{d\text{Ace}_j}{dt} = k_A^{\text{ac}} (1 - \text{Ace}_j) - k_A^{\text{deac}} \text{Ace}_j + k_A^{\text{stress}} S_j (0.5 - \text{Ace}_j), \quad (21)$$

$$\frac{d\text{Acc}_j}{dt} = k_X^{\text{open}} \text{Ace}_j (1 - \text{Acc}_j) - k_X^{\text{close}} \text{Met}_j \text{Acc}_j. \quad (22)$$

Gene-expression potential is a linear combination of the chromatin state variables plus noise,

$$\text{Exp}_j = e_0 + \alpha_A \text{Ace}_j + \alpha_M \text{Met}_j + \alpha_X \text{Acc}_j + \xi_j, \quad \xi_j \sim \mathcal{N}(0, \sigma_E^2), \quad (23)$$

with $\alpha_M < 0$. Instability and plasticity evolve as

$$\frac{d\text{Inst}_j}{dt} = k_I^{\text{base}} + k_I^{\text{stress}} S_j - \kappa_I \text{Inst}_j, \quad (24)$$

$$\frac{d\text{Plas}_j}{dt} = k_\Psi^{\text{base}} + k_\Psi^I \text{Inst}_j - \kappa_\Psi \text{Plas}_j. \quad (25)$$

Stemness Stem_j and differentiation Diff_j are coupled,

$$\frac{d\text{Stem}_j}{dt} = -k_{\Sigma}^{\text{diff}} \text{Diff}_j \text{Stem}_j + k_{\Sigma}^{\Psi} \text{Plas}_j (1 - \text{Stem}_j) - k_{\Sigma}^{\text{stress}} S_j \text{Stem}_j, \quad (26)$$

$$\frac{d\text{Diff}_j}{dt} = k_{\Delta} (1 - \text{Diff}_j) (1 + 2(1 - \text{Stem}_j)) - k_{\Sigma}^{\Psi} \text{Plas}_j \text{Diff}_j. \quad (27)$$

Epigenetic age Age_j accumulates over time and is partially reset by high plasticity,

$$\frac{d\text{Age}_j}{dt} = k_T - k_T^{\text{reset}} \text{Plas}_j \text{Age}_j. \quad (28)$$

Discrete differentiation and chromatin states are assigned by threshold rules applied to Diff_j and Acc_j , respectively.

3.4 Synergy

Dynamics

Let F_j^t , F_j^m , and F_j^e denote tumor, metabolic, and epigenetic fitness in region j .

Combined fitness is

$$F_j = w_t F_j^t + w_m F_j^m + w_e F_j^e, \quad w_t + w_m + w_e = 1. \quad (29)$$

Hypoxia, oxidative stress, and metabolic stress evolve with bounded production and decay,

$$\frac{d\text{Hyp}_j}{dt} = k_H^{\text{resp}} \text{MS}_j (1 - \text{Hyp}_j) - \kappa_H \text{Hyp}_j, \quad (30)$$

$$\frac{d\text{Ox}_j}{dt} = k_O^{\text{resp}} F_j^m (1 - \text{Ox}_j) - \kappa_O \text{Ox}_j, \quad (31)$$

$$\text{MS}_j = \alpha_H \text{Hyp}_j + \alpha_O \text{Ox}_j. \quad (32)$$

Mutation pressure evolves as

$$\frac{d\text{MP}_j}{dt} = k_M^{\text{resp}} (\alpha_M^{\text{met}} \text{MS}_j + \alpha_M^{\text{ox}} \text{Ox}_j) (1 - \text{MP}_j) - \kappa_M \text{MP}_j. \quad (33)$$

Therapy resistance evolves as

$$\frac{dR_j}{dt} = k_R^{\text{resp}} (\alpha_R^{\Psi} \text{Plas}_j + \alpha_R^{\text{res}} \text{Res}_j) (1 - R_j) - \kappa_R R_j, \quad (34)$$

where Res_j is cellular resilience. Adaptive capacity evolves as

$$\frac{d\text{AC}_j}{dt} = k_{\text{AC}}^{\text{resp}} \text{Exp}_j (1 - \text{AC}_j) - \kappa_{\text{AC}} \text{AC}_j. \quad (35)$$

Cellular resilience evolves as

$$\frac{d \text{Res}_j}{dt} = k_{\text{Res}}^{\Sigma} \text{Stem}_j (1 - \text{Res}_j) - k_{\text{Res}}^{\text{stress}} \text{MS}_j \text{Res}_j. \quad (36)$$

System stability evolves as

$$\frac{d \text{Stab}_j}{dt} = k_{\text{Stab}}^{\text{Res}} \text{Res}_j (1 - \text{Stab}_j) - k_{\text{Stab}}^{\text{O}} \text{Ox}_j \text{Stab}_j - k_{\text{Stab}}^{\text{M}} \text{MP}_j \text{Stab}_j. \quad (37)$$

Two additional negative feedbacks are implemented: high resistance reduces selection pressure, and high mutation pressure reduces epigenetic fitness.

3.5 Numerical

Treatment

The three continuous-state engines (Metabolism, Epigenetics, and Synergy) use explicit time-stepping with a configurable step Δt , corresponding to a forward-Euler discretization of the update rules above. Time-step selection follows standard stability considerations: Δt must be small relative to the fastest rate constant in each subsystem. Bounds are enforced per variable class as described at the beginning of this section. The Tumor Evolution Engine is not integrated by Euler stepping: it is a discrete stochastic process in which births, deaths, and mutation opportunities are drawn from Poisson distributions at each step. Development mode is provided for faster execution during testing, but it increases the effective step size and is not recommended for scientific experiments.

4 Software

Architecture

4.1 Module

Organization

ISO is organized as four independent engine packages under a common `engines/` directory, together with a master orchestration script and a project-wide validation script at the repository root. Each engine package contains its own `engine.py`, a set of dedicated validators, and a `validate_all.py` aggregator. Engine packages do not import one another; they communicate only through the master orchestration layer, which executes them as separate processes.

This design has three principal motivations. First, it makes each engine independently testable: an error in one engine cannot silently propagate into another. Second, it makes each engine independently reusable: a researcher interested only in clonal dynamics or only in tumor metabolism can use the corresponding engine on its own. Third, it enforces explicit interfaces: any information that flows between engines must be passed explicitly through orchestration rather than implicitly through shared mutable state.

Table 2: Software architecture components.

Component	Responsibility
<code>engines/tumor_evolution/</code>	Tumor/evolution engine, configuration, results, and validation suite
<code>engines/metabolism/</code>	Metabolism engine, configuration, results, and validation suite
<code>engines/epigenetics/</code>	Epigenetics engine, configuration, results, and validation suite
<code>engines/synergy/</code>	Synergy engine, configuration, results, and validation suite
<code>master_engine.py</code>	Sequential orchestration of the four engines as sub-processes
<code>validate_project.py</code>	Project-wide execution of all engine validation suites

4.2 Configuration and Results

Each engine exposes two primary dataclasses: a `SimulationConfig` that holds all configuration parameters (including a random seed and a validation flag), and a `SimulationResults` object that holds the recorded histories and summary statistics produced by a simulation run. This separation ensures that simulations are fully specified by a configuration object and that results are fully self-describing. Configuration validation is performed at engine construction, and state validation is performed after each recorded step when `strict_validation` is enabled.

4.3 Master Orchestration

The master orchestration script (`master_engine.py`) runs each engine in sequence as a subprocess. It locates the engine directories relative to its own location, captures standard output and standard error, records execution time, and prints a summary. If one engine fails, orchestration continues with the remaining engines and the failure is reported in the summary. This design ensures that a failure in one engine cannot prevent the others from completing, and it preserves the independence of the engines at the level of process execution.

It is important to be precise about what the master orchestration currently does and does not do. The engines are integrated architecturally and conceptually: they share a common design, a common configuration paradigm, and a common validation philosophy. However, the current master script does not automatically pass complete live state trajectories from one engine into the next at each simulation step. Each engine runs to completion on its own. Dynamic, step-by-step cross-engine transfer of state variables is a target of future work (Section 9).

4.4 Project-Wide

Validation

The project-wide validation script (`validate_project.py`) executes each engine's `validate_all.py` validator as a subprocess and produces a unified report. It returns a non-zero exit code if any validation fails, enabling integration with continuous integration systems and reproducibility workflows. As with orchestration, a failure in one validator does not prevent the remaining validators from running.

5 Validation

and

Verification

5.1 Validation

Philosophy

Validation in ISO is directed at the software and its mathematical consistency, not at the biological correctness of any specific empirical claim. We distinguish explicitly among five categories of verification:

1. **Code validation:** ensuring that the implementation matches the documented update rules and that state invariants hold.
2. **Numerical consistency:** ensuring that state variables remain finite, bounded, and free of NaN or infinite values throughout simulation.
3. **Deterministic reproducibility:** ensuring that identical configurations with identical random seeds produce identical recorded numerical histories.
4. **Directional plausibility of implemented relationships:** ensuring that implemented relationships (for example, that higher stress leads to higher mutation pressure within the Synergy Engine) behave in the expected direction under synthetic or perturbed inputs.
5. **Empirical validation:** comparison of simulation output against external biological or clinical data.

ISO currently implements categories (1)–(4). Category (5) is deliberately not claimed, and is a target of future work.

5.2 Engine

Validation

Suites

Each engine includes a set of validators that cover its specific domain. Validation in every engine verifies that state variables remain within their intended bounds, that all recorded values are finite, that array shapes are consistent across the simulation, and that same-seed runs produce identical histories. Beyond this common foundation, engine-specific validators verify the following domain relationships:

- **Tumor Evolution:** monotonic and carrying-capacity-bounded growth in the absence of mutation and death; greater mutation activity and greater clonal diversity under higher mutation rate; correct driver-plus-passenger bookkeeping; validity of parent and child references in mutation records; Shannon and Simpson diversity matching analytical values for synthetic populations; lineage generation and root-ancestor consistency.
- **Metabolism:** reduction of concentration gradients by diffusion; fixation of vascular regions at supply concentration; non-negativity and boundedness of all concentrations; occurrence of oxygen and glucose consumption, ATP production, lactate production, and ROS production; decrease of pH under lactate accumulation; correct assignment of metabolic phenotypes to synthetic states; reproducibility across identical seeds.
- **Epigenetics:** boundedness of all continuous state variables; correct classification of chromatin state for synthetic accessibility values; correct classification of differentiation state for boundary values of the differentiation variable; reproducibility across identical seeds.
- **Synergy:** correctness of the weighted combined-fitness equation; directional response of mutation pressure, therapy resistance, adaptive capacity, stemness, and system stability to their respective drivers; boundedness and finiteness of all state variables; long-simulation stability; reproducibility across identical seeds.

5.3 Project-Wide

Validation

Project-wide validation executes all engine validation suites in sequence and produces a unified report. This is the principal integration-level verification in ISO: it ensures that the four engines, taken together, satisfy their individual correctness criteria when executed as a unified software project. It does not constitute a biological validation of any specific simulated outcome.

Table 3: Representative validation tests implemented across the four engines.

Category	Example test	What it verifies
Code	Combined-fitness equation check	Implementation matches documented weighted sum
Numerical	Boundedness of state variables	Continuous state variables remain within their intended ranges
Numerical	Finiteness of histories	No NaN or Inf values in any recorded history
Deterministic	Same-seed comparison	Identical recorded histories for identical seeds
Directional plausibility	Feedback direction tests	Implemented feedback behaves in the expected direction
Directional plausibility	Phenotype classification	Synthetic states are classified into the correct phenotype

6 Reproducibility

6.1 Deterministic

Execution

Every engine uses a seeded NumPy random generator initialized from the `random_seed` field of its configuration. Given identical configurations and identical seeds, each engine produces identical recorded numerical histories on the same platform. This is verified explicitly by the reproducibility validators in each engine: two runs with the same seed must produce identical `SimulationResults`, and two runs with different seeds must produce different histories whenever stochastic state is present. Cross-platform numerical identity has not been formally tested and is not claimed.

6.2 Configuration and Results as First-Class Artifacts

Because all simulation parameters are held in a single configuration dataclass and all outputs are held in a single results dataclass, a complete simulation can be reproduced from a serialized configuration alone. This makes the framework amenable to scripted parameter sweeps, automated regression testing, and reproducible figure generation, without requiring researchers to reconstruct parameter values from logs or source code.

This practice is consistent with widely adopted guidelines for reproducible computational research ([Sandve et al., 2013](#)) and with the FAIR principles for computational artifacts ([Wilkinson et al., 2016](#)).

6.3 Project-Wide Validation as a Reproducibility Gate

The project-wide validation script provides a single-command reproducibility gate. Running it after any code change verifies that all engines still satisfy their correctness criteria. This design ensures that the framework cannot silently drift away from its validated behavior.

7 Results

Because ISO is a computational framework rather than an empirical biological study, the results reported here are framework-level and methodological. They describe what the released software does, what its validation suites verify, and what kinds of model behavior it produces under representative configurations. They do not constitute evidence about any specific biological or clinical claim.

7.1 Framework

Execution

All four engines execute to completion with their default configurations and produce `SimulationResults` containing the documented histories. The master orchestration script runs the four engines in sequence and reports the status of each. The project-wide validation script runs all engine validation suites and reports an aggregate status. Both scripts return exit codes compatible with continuous integration systems.

7.2 Validation

Outcomes

Every engine validation suite completes successfully with the released version of the framework. This includes the numerical, deterministic, and directional-plausibility tests described in Section 5. In particular:

- **Reproducibility:** Same-seed runs produce identical recorded histories in every engine, and different-seed runs produce different histories whenever stochastic state is present.
- **Boundedness and finiteness:** State variables remain within their intended ranges and remain finite throughout long simulations (including simulations of several hundred steps).
- **Implemented relationships:** Directional tests confirm that implemented feedbacks behave in the expected direction. For example, within the Synergy Engine, increasing metabolic stress increases mutation pressure; increasing mutation pressure reduces epigenetic fitness via the implemented cost term; increasing gene expression increases adaptive capacity; increasing resilience increases therapy resistance and system stability; and increasing oxidative stress reduces resilience.

- **Discrete classification:** Synthetic states are classified into the correct metabolic, chromatin, and differentiation classes by the engines' classification rules.

7.3 Representative Model Behavior

To illustrate the kind of behavior ISO produces, we describe qualitatively the trajectories observed in default-configuration runs. In the Tumor Evolution Engine, starting from a small number of clones, populations grow toward the carrying capacity, mutations produce new lineages, and clones with more favorable birth-to-death ratios gradually increase in representation. In the Metabolism Engine, oxygen and glucose gradients form between vascular and non-vascular regions; glycolytic phenotypes dominate in hypoxic regions; lactate accumulates and lowers pH; and necrotic regions emerge in regions of sustained stress. In the Epigenetics Engine, methylation and acetylation drift toward their equilibrium values under baseline conditions, and diverge in the presence of stress, producing heterogeneous epigenetic states across regions. In the Synergy Engine, combined fitness evolves toward intermediate values rather than saturating at the extremes, reflecting the balance of positive drive and negative feedback built into the update rules.

These behaviors are computational demonstrations of the implemented model dynamics. They are consistent with the qualitative expectations encoded in the model and are not evidence of biological correctness.

7.4 Cross-Engine Coupling

At the software level, the four engines are coupled through the master orchestration layer. The current released version executes the engines in sequence and reports their outputs. Direct coupling of the state variables across engines is provided through the public API of each engine, but is not exercised automatically by the master script. This design allows researchers to build custom integration pipelines without modifying the engines themselves. Systems-level integration at the level of live state variables is a target of future work, as described in Section 9.

8 Discussion

8.1 What the Framework Contributes

The central contribution of ISO is a modular, reproducible architecture for representing interacting cancer-state variables across evolutionary, metabolic, and epigenetic scales.

Each engine implements a well-bounded subset of the biology and is validated independently. The Synergy Engine provides a systems-level aggregation of the resulting fitness components. This design has three concrete benefits. First, it makes

the submodels independently testable, which is important because interactions between submodels can otherwise mask errors in individual components. Second, it makes the submodels independently reusable, so that a researcher interested only in one layer can use that layer without adopting the entire framework. Third, it makes the coupling explicit: the interfaces between engines are documented, and any assumptions about how the layers interact are visible in the code and in this manuscript.

The Synergy Engine specifically provides a mechanism for representing systems-level adaptive properties—mutation pressure, therapy resistance, adaptive capacity, resilience, and stability—as emergent consequences of coupled state rather than as exogenous inputs. This is a meaningful structural choice: it places the burden of explanation on the interactions of state variables rather than on hand-specified outcome variables.

8.2 Computational Synergy versus Clinical Synergy

It is essential to distinguish the sense of “synergy” used in this framework from the clinical and experimental sense of the term. In ISO, synergy refers to computational or model-level interaction between state variables: for example, the influence of metabolic stress on mutation pressure, or the influence of gene expression on adaptive capacity. These are internal model relationships, and their existence is a property of the model, not a claim about any biological system. The framework does not claim, and cannot support, any claim about therapeutic synergy in cells, animals, or patients.

Distinguishing these two senses is important because the clinical sense of synergy implies an empirical result that ISO is not designed to produce.

8.3 Limitations

Several limitations should be stated clearly.

No empirical calibration. The parameters in the current release are biologically motivated but are not fitted to specific datasets. Simulations therefore illustrate model behavior rather than predict specific biological outcomes. Empirical calibration is a target of future work.

Parameter identifiability. Because the models are complex and the parameters interact, many parameter combinations can produce qualitatively similar trajectories. This limits the extent to which simulation output can be used to infer parameter values from observed behavior, and it motivates future sensitivity analysis and parameter inference work.

Simplified spatial structure. The Metabolism and Epigenetics Engines use a one-dimensional spatial grid. This captures the qualitative formation of gradients but does not represent the full geometry of a tumor microenvironment. Extending to two or

three dimensions is possible but is not implemented in the current release.

No immune system. The framework does not currently represent immune cells, immune recognition, or immune-mediated killing. Immune visibility is present as a phenotype trait in the Tumor Evolution Engine, but this is a proxy, not a model of immune dynamics.

No therapeutic intervention. The framework does not currently model the administration of therapy or the pharmacokinetics and pharmacodynamics of drugs. Therapy resistance is represented as a state variable, not as a response to a therapeutic agent.

Coupling assumptions. The Synergy Engine aggregates evolutionary, metabolic, and epigenetic fitness through a weighted sum. This is a modeling choice, not a derived result, and other coupling schemes are possible.

Limited dynamic cross-engine coupling. The engines currently run independently under the master orchestrator. Live state-level transfer between engines at each simulation step is not performed by the current release.

No formal uncertainty quantification. The current release does not include Bayesian parameter inference, formal sensitivity analysis, or credible intervals on model outputs.

No benchmark against established simulators. The framework has not been systematically compared against alternative published models of the same subsystems.

No experimental or clinical validation. The framework has not been validated against experimental measurements or clinical outcomes, and no such claim is made.

Software validation is not biological validation. The validation suites verify that the code behaves as documented and that the implemented relationships have the expected direction. They do not verify that the models are biologically correct. The distinction between these two kinds of validation is essential for interpreting the framework.

8.4 Why Modularity Matters

Modularity is not only a software-engineering convenience; it has scientific consequences. When submodels are independent, they can be validated against different kinds of evidence, replaced with alternative formulations, or reused in different combinations. A single monolithic simulator would obscure these possibilities. The ISO design deliberately makes each engine a standalone artifact so that the framework can grow through addition and refinement rather than through rewriting.

9 Future Work

Several directions follow naturally from the current release.

Empirical calibration. Fitting engine parameters against public cancer datasets—for example, mutation spectra, metabolite concentrations in tumor interstitial fluid, or chromatin accessibility measurements—would connect the model to quantitative empirical anchors. This is the most direct way to move from qualitative to quantitative model behavior.

Patient-specific parameterization. Using patient-derived measurements to initialize engine configurations would allow the framework to represent individual tumors more accurately. This is a prerequisite for any future translational application.

State-level cross-engine coupling. The current release couples engines at the process level through the master orchestrator. A richer integration would pass state variables between engines at each simulation step, allowing feedbacks across the layers to be represented dynamically.

Higher-dimensional spatial models. Extending the metabolic and epigenetic grids to two or three dimensions would allow more realistic tumor geometry and vascular structure to be represented.

Immune system modeling. Adding an immune engine, either as an additional layer within ISO or as an external module that reads ISO outputs, would enable questions about immune-tumor interaction that the current framework cannot address.

Therapeutic intervention. A therapy engine that applies external perturbations to the state variables—for example, reducing metabolic fitness or increasing apoptosis susceptibility—would allow the framework to represent treatment schedules and resistance dynamics in a structured way. This is a modeling step, not an empirical claim about treatment efficacy.

Uncertainty quantification. Rigorous uncertainty quantification, including sensitivity analysis and Bayesian inference over model parameters, would strengthen the interpretive value of the framework.

Multi-omics integration. Incorporating transcriptomic, proteomic, and metabolomic measurements as inputs to the engines would allow the framework to leverage the wealth of public multi-omics data.

Additional benchmarks. Developing a formal benchmark suite that compares ISO output to established models of the same subsystems would provide a stronger basis for methodological comparison.

Hypothesis generation. The framework is well-suited to generating mechanistic hypotheses about coupled cancer-state dynamics. These hypotheses would need to be tested experimentally before they could be used to support biological claims. All of these directions are clearly future work. The current release is deliberately scoped to the four engines described above, with an emphasis on correctness, reproducibility, and interpretability.

10 Conclusion

Integrated Systems Oncology provides a modular, reproducible computational framework for modeling tumor evolution, metabolism, epigenetics, and systems-level adaptive interaction. Each of the four engines is independently implemented, independently validated, and independently reusable, and the framework as a whole is verified by a project-wide validation suite. Each engine is deterministic given a random seed, and reproducibility is verified across identical and differing seeds. The framework explicitly distinguishes software validation from biological validation, and it does not claim clinical or experimental validation. Its primary contribution is methodological: a transparent, extensible substrate for mechanistic computational experiments on interacting cancer-state variables. As empirical calibration, additional spatial resolution, immune system modeling, and therapeutic intervention modeling are added, the framework is positioned to support progressively richer computational studies of cancer as a complex adaptive system.

Declarations

Funding

No external funding was received.

Acknowledgements

The author thanks the open-source scientific Python community for the tools that made this framework possible, and acknowledges the broader literature on computational oncology and cancer systems biology that has shaped its design.

Author Contributions

Dewan Sajid Islam designed the framework, implemented all four engines and their validation suites, and wrote the manuscript.

Conflict of Interest

The author declares no conflict of interest.

Ethics Approval

Not applicable. This work is entirely computational and does not involve human participants, human data, or animal subjects.

Consent for Publication

Not applicable.

Data Availability

No new empirical datasets were generated or analyzed in this work. The framework is designed for computational modeling and simulation; all results reported in this manuscript are outputs of the software itself. Existing public datasets are discussed only as potential resources for future empirical calibration work and were not analyzed

as part of the present release.

Code Availability

The complete Integrated Systems Oncology framework, including all four engines, the master orchestration script, and the full validation suite, is available as open-source software. The repository is publicly available at <https://github.com/Dewan-Sajid-Islam/Integrated-Systems-Oncology>. The software version documented in this manuscript is version 1.0.1. A version-specific archival record is available through Zenodo at <https://doi.org/10.5281/zenodo.21773219>, and a concept DOI that resolves to the latest release is available at <https://doi.org/10.5281/zenodo.21773218>.

References

- Altrock, P. M., Liu, L. L., and Michor, F. The mathematics of cancer: integrating quantitative models. *Nature Reviews Cancer*, 15:730–745, 2015. doi:10.1038/nrc4029.
- Baylin, S. B. and Jones, P. A. Epigenetic determinants of cancer. *Cold Spring Harbor Perspectives in Biology*, 8(9):a019505, 2016. doi:10.1101/cshperspect.a019505.
- Beerenwinkel, N., Schwarz, R. F., Gerstung, M., and Markowetz, F. Cancer evolution: mathematical models and computational inference. *Systematic Biology*, 64(1):e1–e25, 2015. doi:10.1093/sysbio/syu081.
- Hanahan, D. Hallmarks of cancer: new dimensions. *Cancer Discovery*, 12(1):31–46, 2022. doi:10.1158/2159-8290.CD-21-1059.
- Kitano, H. Systems biology: a brief overview. *Science*, 295(5560):1662–1664, 2002. doi:10.1126/science.1069492.
- Markert, E. K. and Vazquez, A. Mathematical models of cancer metabolism. *Cancer & Metabolism*, 3:14, 2015. doi:10.1186/s40170-015-0140-6.
- McGranahan, N. and Swanton, C. Clonal heterogeneity and tumor evolution: past, present, and the future. *Cell*, 168(4):613–628, 2017. doi:10.1016/j.cell.2017.01.018.
- Pavlova, N. N. and Thompson, C. B. The emerging hallmarks of cancer metabolism. *Cell Metabolism*, 23(1):27–47, 2016. doi:10.1016/j.cmet.2015.12.006.
- Pavlova, N. N., Zhu, J., and Thompson, C. B. The hallmarks of cancer metabolism: still emerging. *Cell Metabolism*, 34(3):355–377, 2022. doi:10.1016/j.cmet.2022.01.007.
- Sandve, G. K., Nekrutenko, A., Taylor, J., and Hovig, E. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013. doi:10.1371/journal.pcbi.1003285.

Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J. J., Appleton, G., Axton, M., Baak, A., et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi:10.1038/sdata.2016.18.